

Sofia Lima dos Santos

Paradigmas da Programação - Lista 1

Exercício 1:

```
import java.util.Scanner;

public class contador{
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Digite um número: ");
        int n = scanner.nextInt();

        if (n < 0){
            System.out.print("Inválido. Digite um número
positivo. ");
        } else{
            contador(n);
        }
    }

    public static int contador(int n) {
        System.out.println(n);
        if (n == 0){
            System.out.println("BOOM!");
            return n;
        }
        return contador(n-1);
    }
}
```

Exercício 2:

Considere o exemplo: Em uma sala, há 3 armários com 4 caixas cada um, e precisamos procurar por um único livro que pode estar em qualquer caixa de qualquer armário. O código abaixo demonstra o exemplo utilizando o conceito de Labeled Loop.

0 = caixa vazia

1 = livro na caixa

```
public class labeledLoop {
    public static void main(String[] args) {
        int [][] sala = {
            {0, 0, 0, 0},
            {0, 0, 0, 0},
            {0, 0, 0, 1}
        }
    }
}
```

```

};

loop_m:
for (int m = 0; m < sala.length; m++){
    loop_n:
    for (int n = 0; n < sala[m].length; n++) {
        if (sala[m][n] == 1) {
            System.out.print("Você encontrou o livro! Armário "
+ m + ", Caixa " + n);
            break loop_m;
        } else {
            System.out.println("Armário " + m + ", Caixa " + n + ":
O livro não está aqui!");
        }
    }
}
}
}
}

```

Exercício 3:

Anotações

As anotações são metadados, dados que se referem aos próprios dados.

Elas podem ser associadas a vários elementos do código: classes, métodos, variáveis etc.

É um recurso importante que serve como marcador especial e pode ser usado para validação, integração com outros sistemas, controle de fluxo.

Em java existem várias anotações pré-definidas:

- @Documented
- @Inherited
- @Deprecated
- @FunctionalInterface
- @Override
- @interface
- @SuppressWarnings

Exemplo que usa a anotação @Override:

```

class Veiculo{
    void acelerar(){
        System.out.println("Acelerando");
    }
}

class Carro extends Veiculo {
    @Override
    void acelerar() {

```

```
        System.out.println("Acelerando")
    }
}
```

Nesse exemplo, temos uma classe é uma subclasse que possuem um método com a mesma assinatura. Nesse caso, utilizamos a anotação "@Override" para garantir que estamos sobrescrevendo o método da classe pai.

Exercício 4:

A primeira versão lançada possuía todo o código escrito em assembly, contudo, isso era uma grande limitação, já que o Assembly pode variar de computador para computador. Por esse motivo, quatro anos depois, o sistema foi reescrito, e atualmente tem a maioria do seu código em C.

Exercício 5:

Segundo Sebesta, um programa é considerado confiável se ele opera de acordo com suas especificações em todas as condições. Existem diversas características da linguagem que refletem na confiabilidade dos programas:

- Verificação de tipo: testar os tipos de dados em determinado programa para garantir que eles estejam corretos.
Ex: Uma variável declarada como inteiro não deve armazenar texto.
- Tratamento de Exceção: a capacidade de um programa de interceptar erros em tempo de execução, tomar medidas corretivas e depois continuar.
- *Aliasing*: significa ter dois ou mais nomes diferentes que podem ser usados para acessar a mesma célula de memória. Algumas linguagens restringe bastante o uso do *Aliasing*, por ser uma característica perigosa.
- Legibilidade e facilidade de escrita: Quanto mais fácil for escrever um programa, mais provável é que ele esteja correto. A legibilidade afeta a confiabilidade tanto nas fases de escrita quanto de manutenção do ciclo de vida. Programas que são difíceis de ler são difíceis tanto de escrever quanto de modificar.

Exercício 6:

- Processo mais flexível;
- Depuração mais fácil: já que o código é executado linha por linha, os interpretadores costumam oferecer recursos avançados de depuração, o que permite o desenvolvedor analisar o estado do programa em diferentes pontos de execução, tornando a identificação de erros e, conseqüentemente a correção, mais fácil.
- Mensagem de erro mais precisa;
- Análise de código mais rápida: O interpretador pode analisar o código-fonte mais

rápido do que um compilador, que precisa traduzir todo o código para linguagem de máquina antes da execução. Porém, é importante lembrar que como o código é interpretado linha por linha em tempo de execução, a execução pode ser mais lenta em comparação com linguagens compiladas.

Exercício 7:

(F)

(F)

(V)

(V)